

Browser Automation Agent

線上 Demo	https://browser-automation-agent.zeabur.app/
原始碼	https://github.com/j84077200345-dotcom/Browser-Automation-Agent
技術	Python · Playwright · Anthropic / OpenAI · FastAPI · Docker · Zeabur
開發方式	AI-only 工作流(Claude Code + Agent Skill)

專案摘要

本專案實作一個由 LLM 驅動的瀏覽器自動化代理(agent):使用者給定一個自然語言目標與起始 URL,agent 便以 Playwright 操控 headless Chromium,一步一步地觀察頁面、決定動作、執行操作,直到達成目標為止,並產出可驗證的交付物——結構化報告(report.md)、機器可讀資料(data.json)、每一步的截圖,以及完整的執行記錄。系統以 FastAPI 提供 Web 儀表板,容器化後部署於 Zeabur,對外提供公開網址;LLM 供應商不寫死,可在 Anthropic(Claude)與 OpenAI(GPT)之間切換。

1. 系統需求

1.1 作業共通要求

要求	本專案的落實方式
AI-only 工作流	全程以 Claude Code 開發;另附一個 Claude Code Agent Skill(加分項)
Git 證據	10 個語意化 commit,反映由骨架、核心、Web、測試到部署的完整過程
線上部署	容器化部署至 Zeabur,提供公開網址
文件	README 說明執行/驗證方式、關鍵假設與 AI 工作流的運用
無機密資料	僅使用公開網站與自建程式;API key 只走環境變數,絕不進版控

1.2 功能性需求

- 接收自然語言目標與起始 URL,自動規劃並執行多步驟瀏覽器操作。
- 支援的動作:開啟網址、點擊、輸入文字、捲動、讀取頁面、返回上一頁、完成。
- 產出可驗證交付物:Markdown 報告、結構化 JSON 資料、逐步截圖、執行記錄。
- 提供 Web 儀表板:觸發任務、即時檢視每一步進度與截圖、瀏覽歷史結果。
- 提供一鍵預設任務與命令列(CLI)兩種操作入口,便於示範與驗證。

1.3 非功能性需求

- 可靠性:**面對網站改版與動態 class 名稱時,操作仍具韌性。

- **可驗證性:**每次執行的產物與日誌可被人工與程式化檢驗,且不得捏造內容。
- **可部署性:**可容器化並部署至一般 PaaS 平台,取得公開網址。
- **安全性:**僅操作免登入公開頁面;金鑰不落地、不進版控;公開 URL 設每日觸發上限。
- **可維護性:**模組分層清楚、LLM 供應商可抽換、附自動化測試。

2. 使用技術的選擇

技術	用途	選擇理由
Python 3.11	主要開發語言	生態成熟,Playwright / LLM SDK / FastAPI 皆一級支援
Playwright	瀏覽器自動化	跨瀏覽器、自動等待機制完善、官方提供含瀏覽器的 Docker image,部署最省事
Anthropic / OpenAI SDK	LLM 決策(tool-use)	兩家皆原生支援工具呼叫;以抽象層讓 key 不綁單一供應商
FastAPI + Uvicorn	Web 服務與 API	非同步、輕量,與 agent 的 async 迴圈契合,內建型別驗證
Jinja2	伺服器端模板	與 FastAPI 整合佳,無需前端建置流程即可提供即時儀表板
Docker	容器化	以官方 Playwright image 為基底,瀏覽器與系統相依皆內建
Zeabur	部署平台	支援 Dockerfile 自動部署、注入環境變數與產生公開網域
pytest / pytest-asyncio	測試	簡潔的 async 測試支援,涵蓋單元與離線整合測試

2.1 關鍵技術決策

以下三項決策在開發初期即與需求對齊,並影響整體架構:

- **LLM 驅動 vs. 固定腳本:**採 LLM 逐步規劃動作,而非寫死的腳本。理由:更貼近「agent」的題意,對頁面變動更有韌性;缺點是需要 API key 且有 token 成本,故以步數上限與每日上限控管。
- **多供應商可切換:**抽出 LLMClient 抽象層,以 LLM_PROVIDER 環境變數切換 Anthropic / OpenAI。理由:避免供應商鎖定,手上有哪家 key 就用哪家,對評分亦更具彈性。

- **帶編號元素快照(可靠性關鍵):**每次擷取頁面時以 JavaScript 掃描可見互動元素並標上 data-agent-id 編號,LLM 只需以「編號」指定操作對象,而非提供脆弱的 CSS selector。這同時讓模型看到精簡的 accessibility 視圖,省 token 又提升判斷品質。

3. 系統架構設計

3.1 高層架構



3.2 模組分層

- **app/agent/browser.py:**封裝 Playwright,提供高階動作與「帶編號互動元素快照」。
- **app/agent/agent.py:**供應商無關的 agent 主迴圈,協調觀察、決策與執行。
- **app/agent/llm.py:**LLM 供應商抽象層;中立對話回合 + Anthropic / OpenAI adapter。
- **app/agent/tools.py:**工具定義(中立格式)與分派到瀏覽器動作。
- **app/agent/prompts.py:**系統提示;要求誠實回報、只操作公開頁面。
- **app/agent/logger.py:**執行記錄:run.json / log.jsonl / 截圖 / report.md / data.json。
- **app/runner.py:**run 協調器:背景執行、單一併發、每日上限、歷史查詢、示範 run 植入。
- **app/web/:**FastAPI:儀表板、觸發、即時進度 API、靜態產物、健康檢查。

- **app/config.py**:設定集中管理,從環境變數 / .env 載入(含供應商切換)。

3.3 目錄結構

```

app/
├─ config.py          設定(環境變數 / .env)
├─ tasks.py           預設示範任務(免登入、可重現)
├─ runner.py          RunManager:背景執行、併發控制、示範植入
├─ cli.py             命令列執行入口
├─ agent/
│  ├─ browser.py      Playwright 封裝 + 帶編號元素快照
│  ├─ agent.py        供應商無關的 agent 主迴圈
│  ├─ llm.py          LLM 供應商抽象層 (Anthropic / OpenAI)
│  ├─ tools.py        工具定義與分派
│  ├─ prompts.py      系統提示
│  └─ logger.py       執行記錄與產物
├─ web/
│  ├─ server.py       FastAPI:儀表板 / API / 靜態產物
│  └─ templates/      base / index / run
└─ samples/           內建示範 run(啟動時植入 runs)
Dockerfile · requirements.txt · tests/ · .claude/skills/

```

3.4 核心機制

- **帶編號互動元素快照**:以 `page.evaluate` 執行 JavaScript,篩選出可見的 `a / button / input / [role]` 等元素,標記 `data-agent-id` 編號並回傳精簡清單;動作以 `[data-agent-id='N']` 定位,穩定且省 token。
- **供應商抽象**:維持一組中立的對話回合(`UserTurn / AssistantTurn / ToolResultsTurn`),由各家 adapter 轉譯為 Anthropic tool-use 或 OpenAI function calling 格式,agent 迴圈本身不感知供應商。
- **執行記錄**:每步記錄「思考→動作→觀察」與截圖;`run.json` 為標準狀態檔,Web 前端以輪詢方式即時更新。
- **示範 run 植入**:內建一次真實執行的示範 run,啟動時植入 `runs` 目錄,解決 Zeabur 檔案系統重新部署清空、首次載入無內容的問題,使公開 URL 一載入即有可驗證產物。

4. 開發過程

4.1 AI-only 工作流

全程使用 Claude Code(Anthropic 官方 CLI)進行開發:由 AI agent 規劃架構、撰寫程式,並以「實作 → 執行測試 → 修正」的迴圈推進,同時維持有意義的 Git commit 歷史。開發初期先就三項關鍵決策(任務型態、決策方式、供應商)與需求對齊,再逐一實作各層。此外,將「執行一個瀏覽器自動化目標」的流程封裝為一個 Claude Code Agent Skill,作為可重複呼叫的技能(對應题目的 Skills 加分項)。

4.2 迭代與關鍵轉折

- 先建立骨架與設定,再依 browser → agent → web → 測試 → 部署逐層堆疊,每層皆有測試把關。
- 端對端實測時發現 news.ycombinator.com 於開發網路環境連線逾時,遂改以極穩定的 quotes.toscrape.com 作為示範 run;HN 相關任務保留於預設清單,於雲端環境仍可執行。
- 修正一項前端缺陷:預設任務按鈕原以 inline onclick 帶入,因 tojson 產生的引號會截斷 HTML 屬性而失效,改用 data-* 屬性 + addEventListener 後正常。

4.3 Git Commit 歷史

Commit	日期	說明
4c4f799	06-30	chore: 專案骨架(設定、相依套件、目錄結構)
135ceb8	06-30	feat(agent): Claude tool-use 迴圈 + Playwright + 執行記錄
26a3e9a	06-30	feat(web): FastAPI 儀表板 + run 協調器 + CLI + 預設任務
65c7abe	06-30	test: 單元測試與離線瀏覽器整合測試
714b211	06-30	docs+build: README、Dockerfile(Zeabur)、Claude Code Skill

2fcc1ad	07-01	feat(llm): 多供應商 LLM 抽象層(Anthropic / OpenAI)
0fcd2d8	07-01	feat: 內建示範 run 並於啟動時植入 runs
feedc4f	07-01	build: Dockerfile 以 python -m uvicorn 啟動
27cb3b2	07-01	fix(web): 修正預設任務按鈕無法帶入表單
02d9af6	07-01	docs: README 填入 Zeabur 線上 Demo 公開網址

5. 測試驗證手法

5.1 自動化測試(共 14 項,全數通過)

- **單元測試**:設定解析、RunLogger 產物生命週期、預設任務、工具分派(含 finish)、示範 run 植入。
- **離線瀏覽器整合測試**:以內嵌 HTML(不需網路)驗證帶編號元素快照:正確標記可見元素、忽略隱藏元素、依編號操作與錯誤時的優雅處理。
- **LLM 抽象層測試**:驗證中立對話回合分別轉換為 Anthropic / OpenAI 格式的正确性,以及供應商/模型解析。
- **Web 啟動測試**:以 Starlette TestClient 驗證各路由、API 與錯誤處理(如未設 key 時擋下觸發)。

5.2 端對端驗證

以 OpenAI gpt-4o 實際執行 quotes_humor 任務:agent 於 4 步內完成,正確擷取 humor 標籤的全部 12 則名言(含跨頁),產出 report.md 與 data.json,並存下逐步截圖。此次執行即收錄為專案內建的示範 run。

5.3 可驗證產物

產物	用途
report.md	交付的 Markdown 報告(人可讀)
data.json	結構化擷取資料(可程式化比對)

screenshots/step-NN.png	每一步的截圖(視覺佐證)
run.json	標準狀態檔:目標、狀態、每步摘要、最終結果
log.jsonl	逐步事件記錄(思考 → 動作 → 觀察)

6. 上線網址與使用方式

線上 Demo:<https://browser-automation-agent.zeabur.app/>

健康檢查:<https://browser-automation-agent.zeabur.app/healthz>

6.1 使用方式(Web 儀表板)

- 開啟公開網址,首頁「歷史 run」即可看到內建示範 run 的報告、資料與截圖。
- 點選預設任務按鈕一鍵帶入目標與起始 URL,或自行輸入自訂目標。
- 按「開始執行」後,系統以背景任務執行,run 詳情頁會即時顯示每一步與截圖。
- 完成後於詳情頁檢視渲染後的報告,並可下載 report.md / data.json / run.json。

6.2 API 端點

方法 / 路徑	說明
GET /	儀表板首頁
POST /run	觸發一次新的 run(表單:goal, start_url)
GET /runs/{id}	run 詳情頁(即時輪詢)
GET /api/status	目前忙碌狀態、供應商、每日額度
GET /api/runs	歷史 run 列表(JSON)
GET /api/runs/{id}	單筆 run 完整狀態(JSON)
GET /artifacts/...	靜態提供截圖 / report.md / data.json
GET /healthz	健康檢查

6.3 本機執行

```
# 啟用虛擬環境並安裝
./browser_auto_env/Scripts/Activate.ps1
pip install -r requirements.txt
python -m playwright install chromium

# 設定供應商與金鑰(.env)
copy .env.example .env # 設 LLM_PROVIDER 與對應的 API key
```



```
# 啟動 Web 介面
uvicorn app.web.server:app --reload    # http://localhost:8000

# 或用 CLI 直接執行預設任務
python -m app.cli --preset quotes_humor
```

6.4 部署(Zeabur)

- 以 Dockerfile 部署,base image 為官方 mcr.microsoft.com/playwright/python(內建瀏覽器)。
- 於平台設定環境變數機密:LLM_PROVIDER、對應的 OPENAI_API_KEY 或 ANTHROPIC_API_KEY、以及 DAILY_RUN_LIMIT。
- 平台自動注入 \$PORT 並啟動 uvicorn;產生公開網域即可對外服務。

7. 已知限制與未來改進

- 產物為暫時性;Zeabur 檔案系統於重新部署時清空(已以內建示範 run 緩解)。
- 目前僅以文字(accessibility 視圖)餵給 LLM;未來可加入截圖的視覺輸入以處理純圖形介面。
- 一次僅執行一個 run(避免小型容器 OOM);未來可加入佇列與水平擴充。
- 僅支援免登入公開頁面;需登入之情境(如 NotebookLM)需另行處理認證與合規議題。

8. 結語

本專案在符合作業所有共通要求的前提下,交付了一個可實際運作、可驗證、且已上線的 LLM 驅動瀏覽器自動化代理。透過帶編號元素快照與供應商抽象等設計,兼顧了可靠性與彈性;透過完整的執行記錄與內建示範 run,確保成果可被獨立檢驗。